

Embedded C Cheat Sheet

"A practical reference for bare-metal
and embedded systems programming."

by Matias Back

Bit Manipulation

Context: Most peripherals are configured by setting or clearing specific bits inside control registers.

```
Set bit: REG |= BIT(3);
Clear bit: REG &= ~BIT(3);
Toggle bit: REG ^= BIT(3);
Check bit: if (REG & BIT(3)) { // Bit is high }
```

Examples: Turning LEDs on/off, Enabling UARTs, Starting timers, Configuring interrupts.

Register Fields (Bit Masks)

Context: Some peripherals use multiple bits to represent a value. These groups of bits are called fields.

Write a value to bits 4–6 without affecting neighboring bits:

```
#define FIELD_MASK 0x07U
#define FIELD_POS 4U
REG &= ~(FIELD_MASK << FIELD_POS);
REG |= ((value & FIELD_MASK) << FIELD_POS);
```

Structures

Context: Structures group related data together, making code easier to understand and maintain.

```
typedef struct {
    uint16_t temperature; uint16_t pressure; uint8_t status;
} SensorData_t;
```

Example:

```
SensorData_t sensor; sensor.temperature = 25;
sensor.status = 1;
```

Common uses: Sensor measurements, Communication packets, Device configuration, Register definitions.

Enumerations (State Machines)

Context: Many embedded systems are implemented as state machines. Enumerations provide readable names instead of magic numbers.

```
// Define in the header
typedef enum {
    STATE_INIT, STATE_IDLE, STATE_RUN, STATE_ERROR
} State_t;
```

```
// Usage in the code
switch(state) {
    case STATE_IDLE:
        break;
    case STATE_RUN:
        break;
    default:
        break; }
```

Pointers

Context: Pointers are fundamental in embedded programming because hardware peripherals are accessed through memory addresses.

```
Basic pointer: uint32_t *ptr;
Pointer to constant data: const uint32_t *ptr;
Constant pointer: uint32_t * const ptr = address;
Pointer to volatile data: volatile uint32_t *ptr;
Typical hardware register pointer: volatile uint32_t * const
GPIO = (volatile uint32_t*)0x40020014;
```

Example:

```
uint32_t temperature = 25; uint32_t *temp_ptr = &temperature;
*temp_ptr = 30; // Changes the value to 30
```

Super Loop Architecture

Context: Many bare-metal systems use a simple infinite loop after initialization. The loop continuously reads inputs, processes data, and updates outputs.

```
int main(void) {
    system_init();
    while(1) { // Infinite main loop
        read_inputs();
        process_events();
        update_outputs();
    }
}
```

Memory-Mapped I/O

Context: Microcontroller peripherals appear as memory locations. Reading or writing to specific addresses directly controls hardware. #define goes in the header.

Register Definition & Usage:

```
#define GPIOA_ODR (*(volatile uint32_t*)0x40020014)
GPIOA_ODR |= BIT(5); // Inside e.g. main, pin 5 HIGH
```

Examples of memory-mapped peripherals:
GPIO, UART, SPI, I²C, Timers, ADC.

Register Mapping with Structures

Context: Many MCU vendors use structures to represent peripheral register layouts. #define goes in the header.

```
typedef struct { volatile uint32_t CR; volatile uint32_t SR;
volatile uint32_t DR; } UART_TypeDef;
```

Map the Peripheral & Usage:

```
#define UART1 ((UART_TypeDef*)0x40011000)
UART1->CR |= BIT(0); // Enable UART1
```